

```

MAINING_TIME)
{
    Display_time[z] == Time4input_active;
    if (Display_time[z] > Learn_time[z])
    {
        if ((Display_time[z]-Learn_time[z])<TOLERANCE)
        {
            Learn_time[z]=Display_time[z];
            Phase[z] = VERIFY;
        }
        else if (Display_time[z] < Learn_time[z])
        {
            if ((Learn_time[z]-Display_time[z])<TOLERANCE)
            {
                Learn_time[z]=Display_time[z];
                Phase[z]=VERIFY;
            }
            // display remaining time phase
            // inactive input
            // count learn time
        }
        else if (MS_MODE==SLAVE_MODE)
        {
            if (Serial_timeout > 0)
            {
                Serial_timeout--;
                if (Serial_timeout == 0)
                {
                    Reset_serial_packet();
                }
                if (Serial.available() > 0)
                {
                    Serial_timeout = 10;
                    Serial_data = Serial.read();
                    if (Serial_count == 0)
                    {
                        if (Serial_data == START_BYTE)
                        {
                            Serial_count++;
                            Reset_serial_packet();
                        }
                        else if (Serial_count == 1)
                        {
                            if (Serial_data == START_BYTE)
                            {
                                Serial_count++;
                                Reset_serial_packet();
                            }
                        }
                        else if (Serial_count == 2) // time for active input
                        {
                            Slave_time=Serial_data * 256;
                            Serial_count++;
                        }
                        else if (Serial_count == 3) // remaining time
                        {
                            Slave_time=Slave_time + Serial_data;
                            Serial_count++;
                        }
                        else if (Serial_count == 4) // phase(z)
                        {
                            Slave_phase=Serial_data;
                            Serial_count++;
                        }
                        else if (Serial_count == 5) // color
                        {
                            Color=Serial_data;
                            Reset_serial_packet();
                        }
                    }
                    if (Slave_time == DISPLAY_BLANK)
                    {
                        Blankout_display();
                    }
                    else if (Slave_time == DP_BLINK)
                    {
                        if (Slave_phase == LEARN)
                        {
                            Seg_hun |= DP_ON;
                            if (Slave_phase == VERIFY)
                            {
                                Seg_ten |= DP_ON;
                            }
                            if (Slave_phase == DISPLAY_REMAINING_TIME)
                            {
                                Seg_unit |= DP_ON;
                            }
                        }
                        else if (Slave_time == ERROR_CODE)
                    }
                }
            }
            Seg_hun |= DP_ON;
            Seg_ten |= DP_ON;
            Seg_unit |= DP_ON;
        }
        else if (Slave_time!=768)
        {
            Binary2segment(Slave_time);
        }
    }
    //Act_one_tick_over()
    void Act_one_sec_over ()
    {
        if (MS_MODE == MASTER_MODE)
        {
            if (SELF_TEST_MODE)
            {
                Color=COLOR_RED;
                if (Remaining_time[z] > 1000)
                {
                    SELF_TEST_MODE=0;
                    Blankout_display();
                    Send_packet(DISPLAY_BLANK);
                    for (j=0; j<2; j++)
                    {
                        if (Input_color[j] == ACTIVE_INPUT)
                        {
                            Wait4_inactive[j]=1;
                            Wait4_active[j]=0;
                        }
                        else
                        {
                            Wait4_active[j]=1;
                            Wait4_inactive[j]=0;
                            Phase[j] = LEARN;
                        }
                    }
                }
                else
                {
                    Binary2segment(Remaining_time[z]);
                    Send_packet(Remaining_time[z]);
                    Remaining_time[z]=Remaining_time[z]+1;
                }
            }
            else
            {
                if ((Input_color[0] == ACTIVE_INPUT)
                && (Input_color[1] == ACTIVE_INPUT))
                {
                    Color=COLOR_RED + COLOR_GREEN;
                    Seg_hun |= DP_ON;
                    Seg_ten |= DP_ON;
                    Seg_unit |= DP_ON;
                    Send_packet(ERROR_CODE);
                    return;
                }
                if (Count_learn_time==1)
                {
                    Time4input_active++;
                    if (Display_on==0)
                    {
                        if (TEST_MODE==1)
                        {
                            //Send_packet(Time4input_active);
                            Binary2segment(Time4input_active);
                        }
                        else
                        {
                            Blankout_display();
                        }
                        Send_packet(Time4input_active);
                    }
                }
                else
                {
                    Blankout_display();
                    Send_packet(DISPLAY_BLANK);
                }
            }
            if (Display_on==1)
            {
                if (Remaining_time[z]==0)
                {
                    Display_on=0;
                    Blankout_display();
                    Send_packet(DISPLAY_BLANK);
                }
                else
                {
                    Binary2segment(Remaining_time[z]);
                }
            }
            Send_packet(Remaining_time[z]);
            Remaining_time[z]--;
        }
    }
}

void Act_half_sec_over (void)
{
    if (MS_MODE == MASTER_MODE)
    {
        if (SELF_TEST_MODE==1)
        {
            return;
        }
        if (Red_input == ACTIVE_INPUT) && (Green_input == ACTIVE_INPUT)
        {
            Blankout_display();
            Send_packet(DISPLAY_BLANK);
            return;
        }
        if ((Count_learn_time == 0) && (TEST_MODE==1))
        {
            if (z==0)
            {
                Color=COLOR_RED;
            }
            else if (z==1)
            {
                Color=COLOR_GREEN;
            }
            if (Phase[z] == LEARN)
            {
                Seg_hun |= DP_ON;
            }
            if (Phase[z] == VERIFY)
            {
                Seg_ten |= DP_ON;
            }
            if (Phase[z] == DISPLAY_REMAINING_TIME)
            {
                Seg_unit |= DP_ON;
            }
            if (Count_learn_time == 0)
            {
                Send_packet(DP_BLINK);
            }
        }
        void Send_packet(unsigned int x) //step 4 done
        {
            digitalWrite(RE,DE,HIGH);
            delayMicroseconds(100);
            Serial.print (START_BYTE,BYTE);
            delayMicroseconds(100);
            Serial.print (START_BYTE,BYTE);
            delayMicroseconds(100);
            Serial.print (x/256,BYTE);
            delayMicroseconds(100);
            Serial.print (x%256,BYTE);
            delayMicroseconds(100);
            Serial.print (Phase[z],BYTE); // 1 learn, 2 verify, 3 display remaining time
            delayMicroseconds(100);
            Serial.print (Color,BYTE); // z = 0 for red, 1 for green
            delay(2); // add delay of 1 character before switching off re_de
            digitalWrite(RE,DE,LOW);
        }
        void Blankout_display()
        {
            Seg_hun=FND_BLANK;
            Seg_ten=FND_BLANK;
            Seg_unit=FND_BLANK;
        }
        void Act_setup()
        {
            Remaining_time[0] = 000;
            Grid_no=0;
            Phase[0] = LEARN; //At power on by default goes application in learn mode
            Phase[1] = LEARN;
            Blankout_display();
            digitalWrite(P1,HIGH);
            digitalWrite(P2,HIGH);
            TEST_MODE = digitalRead(P1);
            MS_MODE = digitalRead(P2);
            SELF_TEST_MODE=1;
        }
    }
}

int main(void)
{
    init();
    setup();
    for (;;)
    {
        loop();
        return 0;
    }
}

```